¿Cómo demuestra la Primera Ley de De Morgan que negar una búsqueda disyuntiva equivale a la intersección de las condiciones negadas, y por qué esta equivalencia es clave para optimizar la evaluación lógica y evitar errores en el filtrado de grandes volúmenes de datos?

- La primera ley de De Morgan dice que el complemento de la unión de dos conjuntos es igual a la intersección de sus complementos respectivos; y se demuestra de la siguiente manera:

  $U= \{1,2,3,4,5\}$ $A= \{1,2\}$ $B= \{2,3\}$

  $A \cup B= \{1,2,3\}$

  $(A \cup B)^c= \{4,5\}$

  $(A^c)= \{3,4,5\}$

  $(B^c)= \{1,4,5\}$

  $A^c \cap B^c= \{4,5\}$

  $\therefore$ **$(A \cup B)^c = A^c \cap B^c$**

  Esta equivalencia es clave porque, por ejemplo; en una búsqueda con grandes volúmenes de datos, si se quieren encontrar resultados que no coincidan con el conjunto A o B, siguiendo esta ley, se pueden restringir los resultados a la intersección de lo excluido.

Plantea un escenario científico o computacional donde se requiera filtrar datos no válidos.

Escenario: Filtrado de calificaciones de un examen escolar.

(U)= Conjunto total de registros de calificaciones ingresados al sistema.

$U= \{x1, x2, x3, ..., xn\}$

(A)= Calificaciones fuera de rango

$A= \{x \in U \mid \text{calificación}(x)<0 \vee \text{calificación}(x)>100\}$

(B)= registros incompletos o vacíos

$B= \{x \in U \mid \text{calificación}(x) \text{ es nulo}\}$

Forma disyuntiva negada:

¬(A∨B)

Forma de intersección negada:

¬A∧¬B

Escribe el pseudocódigo (o sintaxis equivalente en Python/SQL) para aplicar ambos filtros sobre la base de datos o arreglo.

```
# Universo (U): Lista total de registros de calificaciones capturados
registros_calificaciones = [85, -5, 100, None, 105, 0, 72, None, 94]

# ---------------------------------------------------------------------

# 1. Filtro mediante Forma Disyuntiva Negada: ¬(A ∨ B)

# "Descartar si es nulo O está fuera del rango [0, 100]"

# ---------------------------------------------------------------------

        datos_validos_disyuntiva = [ c for c in registros_calificaciones if not (c is
None or (c < 0 or c > 100)) ]

# ---------------------------------------------------------------------

# 2. Filtro mediante Forma de Intersección Negada: ¬A ∧ ¬B

# "Conservar si NO es nulo Y es >= 0 Y es <= 100"

# ---------------------------------------------------------------------



        datos_validos_interseccion = [ c for c in registros_calificaciones if (c is not
None) and (c >= 0 and c <= 100) ]

# Verificación de equivalencia

print("Forma disyuntiva negada:   ", datos_validos_disyuntiva)

print("Forma intersección negada: ", datos_validos_interseccion)

print("¿Son idénticos?: ", datos_validos_disyuntiva == datos_validos_interseccion)
```

Demuestra matemáticamente mediante un Diagrama de Venn (o tabla de pertenencia) la equivalencia entre ambos lados de la identidad y explica cuál de las dos formas de filtro es computacionalmente preferible evaluar en un entorno con millones de registros.

	$x \in A$	$x \in B$	$A \cup B$	$(A \cup B)^c$	A^c	B^c	$A^c \cap B^c$
Fuera de rango, no nulo	1	0	1	0	0	1	0
Nulo	0	1	1	0	1	0	0
Ambos errores	1	1	1	0	0	0	0
Datos validos en U	0	0	0	1	1	1	1

En mi opinión, la forma de intersección negada ($\neg A \wedge \neg B$) es computacionalmente preferible porque en esta forma basta con que la primera condición no se cumpla para que el procesador abandone automáticamente la evaluación en el resto de la fila e inmediatamente pase a la siguiente, ahorrando así millones de comparaciones innecesarias.

Bibliografía:

- Leyes de De Morgan. (17 de enero de 2026). En *Wikipedia*. https://es.wikipedia.org/wiki/Leyes_de_De_Morgan
- MATEMÁTICA 1. (20 de octubre de 2021). *CONJUNTOS 16 | Leyes de De Morgan* [Video]. YouTube. https://www.youtube.com/watch?v=pouUHTv99-8
- Lovaglia, F. M., Elmore, M. A., & Conway, D. (2014). *Álgebra*. Editorial Harla.
- Python Software Foundation. (11 de noviembre de 2026). *El tutorial de Python (Versión 3.15.2)* [Documentación de software]. https://docs.python.org/es/3/tutorial/